



ROS通信模型之深度分析



上官

机器人、运动控制、电机驱动技术

关注他

23 人赞同了该文章 ›

轮子既要会用，更要懂得其原理，才有可能造出新轮子。
基于[ROS框架+](#)对ROS通信模型做进一步分析。

术语定义

1. NAME

图形资源的 Name 是 ROS 提供的一个重要的封装机制，每一个资源都有自己的 namespace 并有自己的特定 Name，可以与其他的资源进行共享。通常情况下，资源可以在其 namespace 中创建其他资源并且这些资源可相互访问。通过代码的集成在不同的 namespace 间可实现资源之间的联系。以 namespace 方式封装的命名系统隔离了系统有可能发生的命名冲突。熟悉 C++ 的同学肯定了解 namespace，它们的作用相当。

2. [Computation Graph](#)⁺

Computation Graph是一个点对点的网络，连接无数的节点，并且交互话题提供服务等，拓扑结构为网状结构。

3. Node

一个节点是一个执行计算的进程。无数节点构成Graph，通过数据流话题、RPC服务以及参数服务相互之间通信。为了提高代码复用率，使得系统功能划分更具细粒度，通常一个机器人控制系统由很多不同功能的节点组成，比如一个节点控制激光扫描，一个节点控制机器人的轮子转动，一个节点完成路径规划等等。这样的节点设计使得整个系统更具模块化，也具备一定的容错能力。

4. [Master node](#)⁺

主节点为系统其它节点提供命名和注册服务，它跟踪系统的话题发布和订阅、服务以及参数功能。主节点用于定位其它的节点，并将这些信息通知给其他的节点并建立他们之间的点对点通信。

5. Message

通过在话题中发布消息来实现节点之间的通信。一个消息是一个简单的数据结构，支持标准原始类型 float、integer、boolean 等。一个消息可以嵌套数组或者结构体。

6. [Topic](#)⁺

遵循异步的发布-订阅机制，话题在节点之间通过消息交互信息。这种机制使得节点之间互相解耦，相互之间不知道对方的存在。节点不关心通信的对方是谁，它们只关心自己发布的话题以及需要的数据。话题可以有多个 Publisher 和 Subscriber，节点从需要的话题中取得消息。话



主节点不会强制 Publisher 的类型一致，但 ROS 客户端将会检查 MD5 是否匹配。

7. Service

service 用于数据流的请求和响应，用于节点之间的同步信息交换，ROS 不检查系统中的 services 重名，如果有多个重名的 service 存在，只有最后一个注册的 service 起作用。

8. Parameter

ROS 系统运行所定义的全局变量，它由 master 节点的 parameter server 负责维护。ROS 的 namespace 使得参数的命名具备非常清晰的层级结构，避免他们之间的冲突。

9. Publisher

publisher 生成指定的话题，将 TCPROS 消息发布给所有的 subscriber

10. Subscriber

订阅指定的话题，从 publisher 那里接收指定的 TCPROS 的消息。

11. Parameter server⁺

参数服务是一个可以通过网络 API 访问的、共享的、多变量词典，节点使用参数服务来实时的保存或者检索参数。常用于静态的、非二进制的的数据，比如系统的配置参数等。它是全局可见的，因此可以运行时修改。参数服务基于 XML-RPC⁺ 实现。

12. API

应用调用接口，用于其他进程的调用。

13. URI

用于唯一的定位一个节点地址，格式 protocol://host:port，protocol 一般是 http 或 rosrpc，host 一般为主机名字或者主机的 IP，port 一般为进程通信的端口号

14. Remote procedure call

远程主机之间的通信技术。

15. XML-RPC

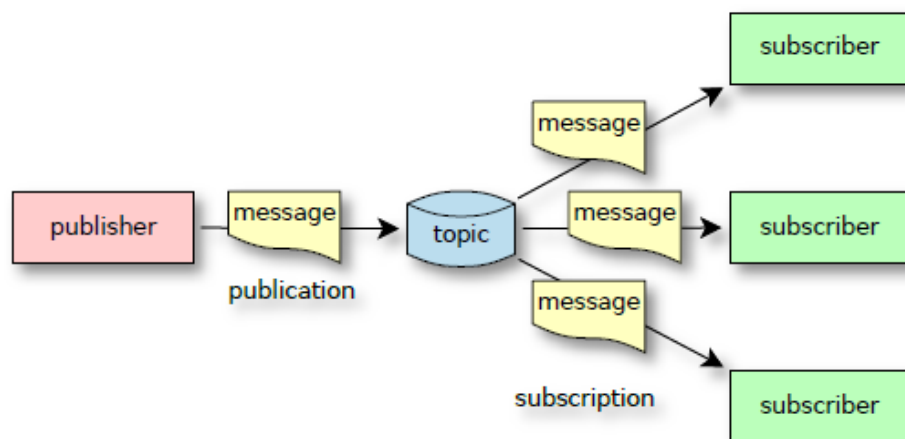
可移植的基于 XML 协议的远程过程调用。

16. TCPROS

基于 TCP 协议的话题与服务的通信数据流。

概述

ROS 通信层是 ros_comm 栈的一部分，遵循 publish/subscribe 模型。如下图所示：



整个 ROS 网络也叫作 computation graph 由不同的包括物理的或者虚拟的、独立命名的节点组成。一个节点就是一个进程，负责从输入端收集数据、执行计算，生成数据并输出。每个节点实现特定的功能并且不依赖于其他节点的存在，纯粹的模块化设计。一个节点可以发布一些话题，其他的节点可以订阅这个话题；services 的机制也类似；话题和服务被设计成为一个固定的类型，通过 ROS 封装之后发布。

假如有一个简单的自动探测器，配备有两个马达，一个距离传感器和一个运行 ROS 的主机。可以将每个传感器或者轮子设计成一个节点，运行 ROS 的主机作为 brain 节点，这就组成了一个 ROS 网络。假定其中一个传感器节点的名字为 sensor_X，它发布的话题为 sensor_X/distance，它以固定的频率采集传感器数据，并将它转换成符合 sensor_X/distance 话题格式的数据，发布到 ROS 网络中。同时 wheel_Y 节点订阅名字为

题，并且发布所有的 `wheel_Y/speed` 话题。它可以根据距离来做一些决策，比如移动机器人用以避开障碍物。

ROS 系统中存在一些全局的声明 `services`，也存在一些 `parameters`。比如，`brain` 节点发布 `sleeping` 服务，如果机器人处于空闲状态，`sleeping` 返回 `true`，反之返回 `false`。此时节点 `sensor_X` 和节点 `wheel_Y` 可以调用 `sleeping` 服务，如果响应为 `true` 则关闭他们的硬件。并且节点 `wheel_Y` 可以调用全局参数 `max_speed` 来调节轮子的速度。

Master 节点作为系统调度中心管理着系统中的 `publisher` 和 `subscribers`。它跟踪发布和订阅的话题和服务，并且提供全局参数服务。

ROS 通过远程过程调用交换 XML-RPC 信息来实现与 Master 节点的通信，在节点之间的话题和服务数据流被编码成自定义的 TCPROS 协议，ROS 的连接地址符合 URI 格式。

远程过程调用

ROS通过远程过程调用来建立节点之间的连接以及数据传输，ROS中的远程过程调用是通过 XML-RPC 实现的。XML-RPC 是 XML 规范的一个子集。这些调用用于管理 `computation graph` 的状态和一些全局设定。XML-RPC 不能用于实际的数据流传输，实际的数据流使用 TCPROS 或者

UDPROS 实现。由于 XML-RPC 的广泛的语言支持，所以被 ROS 开发者选用。以 XML 为基础使得调试简单，它的文本特性使得它独立于传输层编码格式。它以文本内容存在并封装于 HTTP 协议进行传输。并且 XML-RPC 的调用是无状态的，简化了逻辑控制。XML-RPC 的 XML 标签严格遵循语法规则，它比 XML 更易用并且具备 XML 大部分的优势。

- 角色

每一个 ROS 节点具备一个 XML-RPC 服务。它们调用的方式是不同的，ROS 中不同的节点具备不同的 API。Master 节点作为一个独立的节点具备 Master API 的作用，比如话题和服务的注册或注销。Slave API 的声明方式由子节点来管理，比如话题数据流的请求、建立或关闭等命令。Parameter Server API 用于独立的参数服务，管理全局共享变量目录。

• 数据类型

1. `string` `string` 是ASCII字符串类型，它是系统默认的数据类型。
2. `int` or `i4` 是一个32位的有符号整形类型
3. `boolean` 布尔类型，0或1
4. `double` 有符号实数类型
5. `dateTime.iso8601` 遵循ISO-8601格式的时间日期类型
6. `base64` 基于Base64算法编码的二进制字符串
7. `array` 带时间戳的数据数据表
8. `struct` 也称为map，是一个集合

• 请求和响应

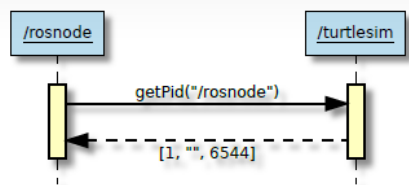
远程过程调用包括两个阶段，一个是请求阶段 `request`，一个是响应阶段 `response`。客户端 `client` 发送请求方式到服务器端 `server`，服务器端接受这个请求并进行处理，处理完成后返回一个响应，用于标记请求的结果成功，如果请求失败则返回一个错误。

一个请求是由用 `methodName` 字符串来命名的 `methodCall` 实体开始的。根据这个方法的名字，一个参数实体遍历一个所有的子参数所包含的数据。参数链表依赖于 API 手册。通常情况下，在 `computation graph` 中第一个参数是调用者节点的名字。

一个响应包含远程调用方式的返回值。ROS 系统中返回值是一个数据表。第一个是状态码，-1 表示错误，0 表示失败，1 表示成功；第二个是状态字符串，一个可读的带有状态码说明的文本，它是可选的，可以为空。第三个是返回值，包含实际的 XML-RPC 数据返回值，表示由指定的 API 定义。

如果响应是一个 XML-RPC 错误，它意味着有一个 `hard error`。

下面是一个 `getPid()` 基于 XML-RPC 的调用，

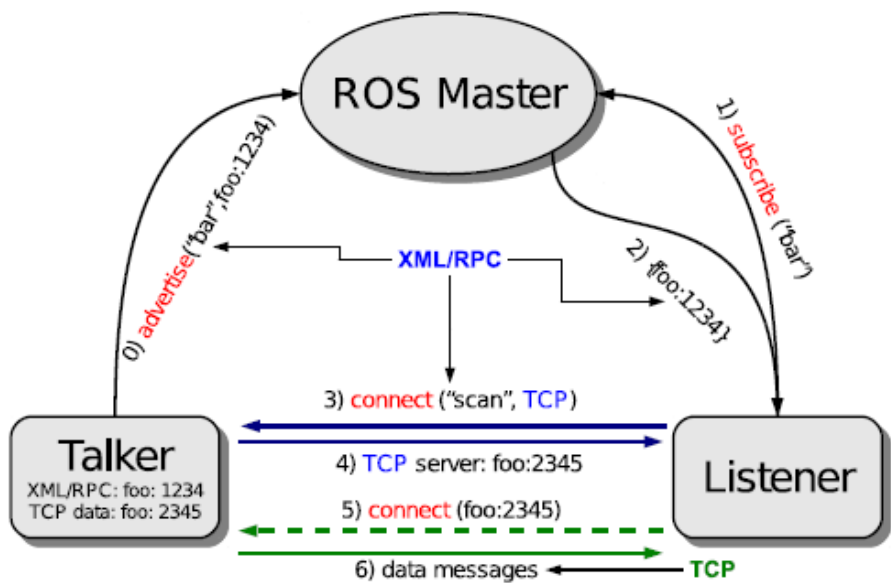


连接模型

一个节点需要与其他节点进行通信才有存在的意义，单独的一个节点是没有意义的。下图是一个常用的通讯模型，一个节点连接到另一个节点，最终请求话题数据流。

一个节点注册它的发布或订阅的话题到 master 节点，每一个话题 publisher 通过 registerPublisher() 的远程过程调用注册到 Master 节点，每一个话题的 subscriber 通过 registerSubscriber() 的远程过程调用注册到 Master 。Master 通过同一个话题连接发布者 和订阅者。

当一个节点向 master 注册它的订阅话题成功时， master 会返回一个包含发布者 URIs 信息的应答。因此 subscriber 和 publisher 就可以建立连接，之后就可以传输数据。当一个新的 publisher 注册到 master 后， master 会启动 publisherUpdate() 通知所有的 subscriber 更新可用的 publisher 的 URI 列表，之后数据流就可以通过 TCPROS 进行交换。

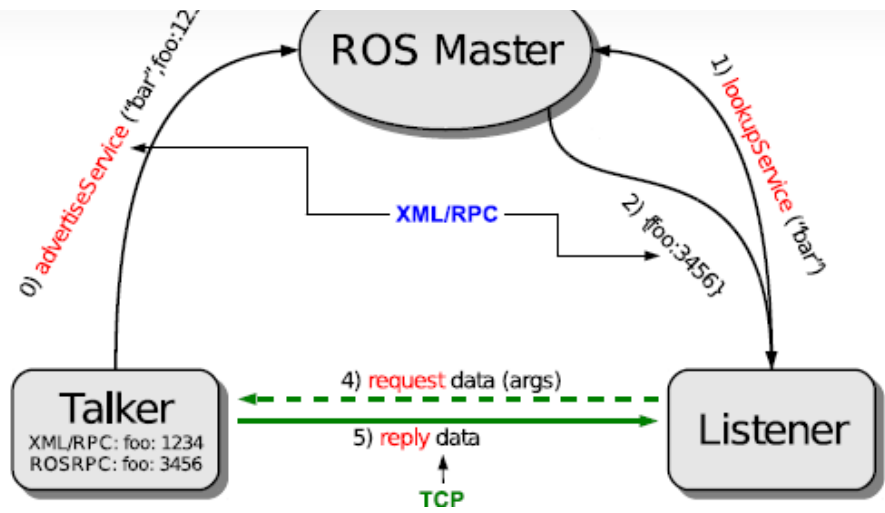


与之对应的是注销过程。publisher 通过 unregisterPublisher() 远程过程调用通知 master 注销自己。subscribers 通过 unregisterSubscriber 远程过程调用通知 master 注销自己。是否关闭任何已建立的数据流取决于 publisher 和 subscriber 本身。

Services 工作方式与话题略有不同，同一个服务能被多个节点注册，但只有最后一个起作用。节点在调用 service 时，通过 lookupService 远程过程调用，在 master 处查找与之对应的 service 的 URI 。之后通过相应的请求消息来通知 service 提供方。service 提供方处理这个请求，如果请求成功会返回一个应答，如果失败则会返回一个错误信息。这些消息都是通过头部附加成功与否的标志信息的 TCPROS 协议实现的。

数据流

虽然 XML-RPC 提供了一个简单清晰的协议用于远程过程调用，但是它的冗长及文本编码方式使得它不适合高带宽和低延迟的任务。ROS 有自己定义的二进制数据流传输协议TCPROS，它传输的原始数据几乎不需要解析，这种设计方式减少了延迟但却增加了带宽。



• 类型描述信息块

数据流是基于某一特定类型的信息交换，这种类型在话题和服务注册时就被指定了。一个消息是一串有序的数据，它的顺序定义由类型描述信息块决定。ROS 中存在两种类型描述信息块，一个是话题描述信息块，一个是服务描述信息块。

话题类型描述信息块是一个纯文本文件，类型名与文件名一致，以 .msg 为文件后缀。每一行可以定义一个强类型变量，也可以是另外一个类型描述信息块，因此消息可以被嵌套。

服务类型描述信息块，由于话题是单向传输的，因此只需要定义一种消息类型，服务是双向的，因为它同时包含请求和响应。服务类型描述信息块的存储类似于话题类型的存储，只不过它的后缀以 .srv 结尾，描述信息块分为两部分，一部分是请求信息描述块，另一部分是响应消息描述块。

哈希 由于描述信息的传递以文本方式在不同的节点之间进行，为了保证消息的一致性，ROS 以 MD5 实现信息的校验，如果双方的 MD5 一致，那么可以判断双方的信息一致。

消息结构

ROS 中的消息以 32 位小端模式存储，它适用于大部分的 x86 以及 ARM 体系。

话题消息的结构是 length+content 格式，length 为 32 位无符号整形，紧接着是内容。

服务消息分请求和响应两种。请求格式与话题格式类似，响应消息则需要附带实际的响应值或者一个错误的消息。

连接头 一旦话题或服务客户端和响应的服务器建立连接，他们必须对所传输的数据流参数达成一致。包括话题或服务名字和类型，可能的多服务请求以及带宽优化等。所有这些参数都是由连接头设定。

Field	Description	Request		Response		
		Tpc	Srv	Tpc	Srv	Err
callerid	Sender Node name	✓	✓	(✓)	✓	(✓)
topic	Topic name	✓				
service	Service name		✓			
md5sum	MD5 sum of the type	✓*	✓*	✓	✓	(✓)
type	Message type	✓*	✓*	✓	✓	(✓)
request_type	Service request type		✓*		✓	
response_type	Service response type		✓*		✓	
message_definition	.msg/.srv contents	(✓)	(✓)	(✓)	(✓)	(✓)
persistent	Persistent service		(✓)			
latching	Latched values			(✓)		
tcp_nodelay	Nagle alg. disabled	(✓)				
error	Error text					✓

ROS 通信的本质是基于 TCP/IP 的 socket 通信，它有一定的优势，但也存在一些实时性问题，虽然官方已经推出了 ROS2.0 版本，但 ROS1.0 强大的生态系统，相信在一定时期内 ROS1.0 还会继续存在，使用一种框架，更要懂得其原理，掌握它的优势，了解它的弊端，这样才有可能造出性能优异的轮子。

参考

TCPROS [A lightweight Opensource communication framework for native integration of resource constrained robotics devices with ROS](#)

编辑于 2018-05-24 09:04

机器人操作平台 (ROS) 同时定位和地图构建 (SLAM) 人工智能

▲ 赞同 23 ▼ ● 添加评论 ↗ 分享 ❤ 喜欢 ★ 收藏 📄 申请转载 ...



理性发言，友善互动



还没有评论，发表第一个评论吧

推荐阅读

ROS学习笔记（八）：ROS通信架构

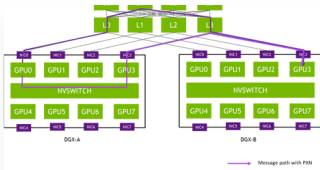
本章主要介绍了通信架构的基础通信方式和相关概念。其中首先介绍了最小的进程单元节点Node,和节点管理器Node master。了解了ROS中的进程都是由很多的Node组成，并且由Node master来管理这些...

少云清 发表于ROS



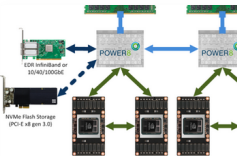
五种网络IO模型和select/epoll对比

慕课网



一文讲清 NCCL 集合通信原理与优化

Tim在路... 发表于大模型



【AI系统】分布式通信与NVLink

ZOMI酱 发表